

R Programming Practice (Part 5): Functions — Additional Challenges

Tim Irwin

2026-04-27

R Programming Practice (Part 5): Functions — Additional Challenges

These are bonus challenges that build on the Functions practice (Part 5). They reuse some of the functions you already wrote — `vec_summary`, `classify_all`, `analyze_column` — and extend them in new directions. Each challenge introduces a new wrinkle: input validation, multiple NULL defaults, building larger pipelines, and more.

The challenges are roughly ordered from easier to harder. There are usually multiple right approaches — focus on writing clear, working code.

```
rm(list = ls())
```

Datasets used

- `airquality` — Daily air quality measurements in New York, May–September 1973.
- `mtcars` — Fuel consumption and 10 aspects of design/performance for 32 cars.
- `iris` — Measurements of 150 iris flowers across 3 species.

Challenge 1: Letter grade converter

Write `letter_grade(score)` that takes a single numeric score (0–100) and returns the corresponding letter:

- 90 or above → "A"
- 80–89 → "B"
- 70–79 → "C"
- 60–69 → "D"
- Below 60 → "F"

Then write `letter_grade_all(scores)` that vectorizes it — takes a vector of scores and returns a vector of letters. Apply it to `c(95, 73, 88, 51, 67, 100, 79)`.

```
### got it right
letter_grade <- function(score) {
  if (score >= 90) {
    "A"
  } else if (score >= 80 & score <= 89) {
    "B"
  } else if (score >= 70 & score <= 79) {
    "C"
  } else if (score >= 60 & score <= 69) {
```

```

    "D"
  } else {
    "F"
  }
}

letter_grades_all <- function(scores) {
  results <- character(length(scores)) #pre-allocate result vector, same length
# as vector input
  for (i in seq_along(scores)) { # loops through index positions (1, 2, 3...)
    # of the scores vector
    results[i] <- letter_grade(scores[i]) # grabs the ith score and passes it to
    # letter_grade function and stores the result in the ith position
  }
  results
}
letter_grades_all(c(95, 73, 88, 51, 67, 100, 79))

## [1] "A" "C" "B" "F" "D" "A" "C"

```

Challenge 2: Tax bracket calculator

Write `calculate_tax(income)` that computes total tax owed using these (simplified) progressive brackets:

- The first \$10,000 of income is taxed at 10%
- Income from \$10,001 to \$50,000 is taxed at 20%
- Income above \$50,000 is taxed at 30%

So for an income of \$60,000, you'd owe: $10000 * 0.10 + 40000 * 0.20 + 10000 * 0.30 = 12000$.

The trick is that *each portion* of income is taxed at a different rate — not the whole amount at the highest rate. Use `if/else if` to handle the three cases.

Test with incomes of 5000, 30000, 60000, and 100000.

Tip: Think about what tax is owed in each case and break the math into pieces. For income above \$50k, you owe the full tax on the first \$10k bracket, the full tax on the \$10k–\$50k bracket, and 30% on whatever is above \$50k.

```

#### got it right
calculate_tax <- function(income) {
  if (income <= 10000) {
    liability <- income * 0.1
  } else if (income > 10000 & income <= 50000) {
    remainder <- income - 10000
    liability <- (10000 * 0.1) + (remainder * 0.2)
  } else {
    surplus <- income - 50000
    liability <- (10000 * 0.1) + (40000 * 0.2) + (surplus * 0.3)
  }
  liability
}

calculate_tax(5000)

## [1] 500

```

```
calculate_tax(30000)
```

```
## [1] 5000
```

```
calculate_tax(60000)
```

```
## [1] 12000
```

```
calculate_tax(100000)
```

```
## [1] 24000
```

Challenge 3: Range filter with two bounded defaults

Write `filter_range(df, col, low = NULL, high = NULL)` that returns the subset of rows where the value in `col` is between `low` and `high` (inclusive).

The twist: **either or both** bounds can be `NULL`. Handle all four cases:

- Both `NULL` → return all rows (no filtering)
- Only `low = NULL` → return rows where `col <= high`
- Only `high = NULL` → return rows where `col >= low`
- Both supplied → return rows where `low <= col <= high`

Test on `mtcars` with "mpg":

1. `filter_range(mtcars, "mpg", low = 20, high = 25)` — cars with mpg between 20 and 25
2. `filter_range(mtcars, "mpg", low = 25)` — cars with mpg 25 or higher (no upper bound)
3. `filter_range(mtcars, "mpg", high = 15)` — cars with mpg 15 or lower (no lower bound)

ORIGINAL

```
# filter_range <- function(df, col, low = NULL, high = NULL) {  
#   if (is.null(low) & is.null(high)) {  
#     df[df[[col]]]  
#   } else if (is.null(low) & !is.null(high)) {  
#     df[df[[col]] <= high, ]  
#   } else if (!is.null(low) & is.null(high)) {  
#     df[df[[col]] >= low, ]  
#   } else if (!is.null(low) & !is.null(high)) {  
#     df[df[[col]] >= low, ] & df[df[[col]] <= high, ]  
#   }  
# }
```

CORRECTED

```
# function takes a data frame, column name, and optional low/high bounds  
# low and high default to NULL so the caller can omit either or both  
filter_range <- function(df, col, low = NULL, high = NULL) {  
  # is.null() checks if the argument was left empty  
  # & means both conditions must be TRUE  
  if (is.null(low) & is.null(high)) {  
    df # no filtering needed, return the whole data frame as-is  
  } else if (is.null(low) & !is.null(high)) {  
    # !is.null(high) means high was provided  
    # df[[col]] pulls the column as a vector  
    # <= high creates a logical vector (TRUE/FALSE for each row)  
    # df[logical_vector, ] keeps only rows where TRUE  
    # the comma is needed to say "all columns"  
  }
```

```

    df[df[[col]] <= high, ]
  } else if (!is.null(low) & is.null(high)) {
    # same idea but filtering for values >= low
    df[df[[col]] >= low, ]
  } else if (!is.null(low) & !is.null(high)) {
    # & combines two conditions: must be >= low AND <= high
    # both conditions go inside the same brackets
    df[df[[col]] >= low & df[[col]] <= high, ]
  }
}

filter_range(mtcars, "mpg", low = 20, high = 25)

```

```

##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Mazda RX4      21.0   6 160.0 110 3.90 2.620 16.46 0  1    4    4
## Mazda RX4 Wag  21.0   6 160.0 110 3.90 2.875 17.02 0  1    4    4
## Datsun 710     22.8   4 108.0  93 3.85 2.320 18.61 1  1    4    1
## Hornet 4 Drive 21.4   6 258.0 110 3.08 3.215 19.44 1  0    3    1
## Merc 240D      24.4   4 146.7  62 3.69 3.190 20.00 1  0    4    2
## Merc 230       22.8   4 140.8  95 3.92 3.150 22.90 1  0    4    2
## Toyota Corona  21.5   4 120.1  97 3.70 2.465 20.01 1  0    3    1
## Volvo 142E     21.4   4 121.0 109 4.11 2.780 18.60 1  1    4    2

```

```
filter_range(mtcars, "mpg", low = 25)
```

```

##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Fiat 128      32.4   4  78.7  66 4.08 2.200 19.47 1  1    4    1
## Honda Civic   30.4   4  75.7  52 4.93 1.615 18.52 1  1    4    2
## Toyota Corolla 33.9   4  71.1  65 4.22 1.835 19.90 1  1    4    1
## Fiat X1-9     27.3   4  79.0  66 4.08 1.935 18.90 1  1    4    1
## Porsche 914-2 26.0   4 120.3  91 4.43 2.140 16.70 0  1    5    2
## Lotus Europa  30.4   4  95.1 113 3.77 1.513 16.90 1  1    5    2

```

```
filter_range(mtcars, "mpg", high = 15)
```

```

##           mpg cyl  disp  hp drat   wt  qsec vs am gear carb
## Duster 360    14.3   8 360 245 3.21 3.570 15.84 0  0    3    4
## Cadillac Fleetwood 10.4   8 472 205 2.93 5.250 17.98 0  0    3    4
## Lincoln Continental 10.4   8 460 215 3.00 5.424 17.82 0  0    3    4
## Chrysler Imperial 14.7   8 440 230 3.23 5.345 17.42 0  0    3    4
## Camaro Z28     13.3   8 350 245 3.73 3.840 15.41 0  0    3    4
## Maserati Bora  15.0   8 301 335 3.54 3.570 14.60 0  1    5    8

```

Challenge 4: Input validation

Write `safe_column_mean(df, col)` that returns the mean of `col` (NAs removed), but **first validates the inputs**:

1. If `df` is not a data frame, stop with "df must be a data frame".
2. If `col` is not a single character string, stop with "col must be a single column name".
3. If `col` is not actually a column of `df`, stop with a message like "Column 'banana' not found in df" (showing the actual column name they passed).
4. If the column exists but isn't numeric, stop with "Column 'X' must be numeric".

If all checks pass, return the mean.

Test it with valid inputs (`safe_column_mean(mtcars, "mpg")`) and at least two invalid inputs to confirm the right errors fire.

Tip: `is.data.frame(df)` checks if something is a data frame. `is.character(col) && length(col) == 1` checks for a single string. `col %in% colnames(df)` checks if the column name exists. `paste0()` lets you build messages with the actual value the caller passed.

MY ORIGINAL

```
# safe_column_mean <- function(df, col) {
#   if (!is.data.frame(df)) {
#     stop("df must be a dataframe")
#   }
#   if (!is.character(col) && !length(col) == 1) {
#     stop("col must be a single column name")
#   }
#   if (!col %in% colnames(df)) {
#     stop("Column", paste0(col), "not found in the df")
#   }
#   if (!is.numeric(df)) {
#     stop("Column", paste0(col), "must be numeric")
#   }
#   mean(col, na.rm = TRUE)
# }
```

CORRECTED

```
safe_column_mean <- function(df, col) {
  # check 1: is.data.frame() returns TRUE/FALSE
  # ! negates it - so this reads "if df is NOT a data frame, stop"
  if (!is.data.frame(df)) {
    stop("df must be a dataframe")
  }
  # check 2: col should be a character string AND length 1 (a single name, not a vector of names)
  # && is used here instead of & - && is better for single TRUE/FALSE checks in if statements
  # wrap the whole condition in !() to negate it - NOT just the first part
  # WRONG: !is.character(col) && !length(col) == 1 - this means BOTH must fail
  # RIGHT: !(is.character(col) && length(col) == 1) - this means if EITHER fails, stop
  if (!(is.character(col) && length(col) == 1)) {
    stop("col must be a single column name")
  }
  # check 3: %in% checks if col is one of the column names in df
  # need parentheses around the whole thing: !(col %in% colnames(df))
  # WRONG: !col %in% colnames(df) - the ! only negates col, not the whole expression
  if (!(col %in% colnames(df))) {
    stop("Column", paste0(col), "not found in the df")
  }
  # check 4: check the actual column, not the whole data frame
  # df[[col]] pulls the column as a vector - then is.numeric() tests it
  # WRONG: !is.numeric(df) - this checks the whole data frame
  # RIGHT: !is.numeric(df[[col]]) - this checks just the column
  if (!is.numeric(df[[col]])) {
    stop("Column", paste0(col), "must be numeric")
  }
  # all checks passed - safe to compute the mean
  # must use df[[col]] to get the actual values, not just col (which is a string)
  # WRONG: mean(col, na.rm = TRUE) - col is just the name "mpg", not the data
}
```

```

# RIGHT: mean(df[[col]], na.rm = TRUE) - this gets the actual numbers
mean(df[[col]], na.rm = TRUE)
}

safe_column_mean(mtcars, "mpg")          # should return 20.09062

## [1] 20.09062

safe_column_mean("not a df", "mpg")      # should stop: df must be a dataframe

## Error in 'safe_column_mean()':
## ! df must be a dataframe

safe_column_mean(mtcars, 123)            # should stop: col must be a single column name

## Error in 'safe_column_mean()':
## ! col must be a single column name

safe_column_mean(mtcars, "banana")       # should stop: Column banana not found

## Error in 'safe_column_mean()':
## ! Columnbanana not found in the df

safe_column_mean(iris, "Species")        # should stop: Column Species must be numeric

## Error in 'safe_column_mean()':
## ! ColumnSpecies must be numeric

```

Challenge 5: Flexible conditional counter

Write `count_matching(x, threshold, comparison = "greater")` that counts how many values in `x` match a condition relative to `threshold`. The `comparison` argument controls what kind of comparison:

- "greater" → count values > threshold
- "less" → count values < threshold
- "equal" → count values == threshold
- "at_least" → count values >= threshold
- "at_most" → count values <= threshold

If `comparison` is anything else, stop with an informative error.

Handle NAs by always ignoring them (don't count NAs as matches, and don't error out on them).

Test all five comparisons on `mtcars$mpg` with `threshold = 20`. You should get five different counts.

```

##### ORIGINAL
# count_matching <- function(x, threshold, comparison = "greater") {
#   count <- 0
#   if (comparison == "greater") {
#     count > threshold
#   } else if (comparison == "less") {
#     count < threshold
#   } else if (comparison == "equal") {
#     count == threshold
#   } else if (comaparison == "at_least") {
#     count >= threshold
#   } else if (comparison == "at_most") {

```

```

#     count <= threshold
#   } else {
#     stop("comparison error")
#   }
#
# }

##### CORRECTED
# takes a vector x, a threshold value, and a comparison type (defaults to "greater")
count_matching <- function(x, threshold, comparison = "greater") {
  # check what type of comparison the caller wants
  # each branch compares every element of x against threshold (vectorized)
  # x > threshold returns a logical vector (TRUE/FALSE for each element)
  # sum() counts the TRUEs - since TRUE = 1 and FALSE = 0
  # na.rm = TRUE tells sum() to skip any NAs instead of returning NA
  if (comparison == "greater") {
    sum(x > threshold, na.rm = TRUE)
  } else if (comparison == "less") {
    sum(x < threshold, na.rm = TRUE)
  } else if (comparison == "equal") {
    sum(x == threshold, na.rm = TRUE)
  } else if (comparison == "at_least") {
    sum(x >= threshold, na.rm = TRUE)
  } else if (comparison == "at_most") {
    sum(x <= threshold, na.rm = TRUE)
  } else {
    # if comparison doesn't match any valid option, stop with an error
    # this catches typos or invalid inputs like "bigger" or "more"
    stop("comparison error")
  }
}

count_matching(mtcars$mpg, 20, "greater")

## [1] 14

count_matching(mtcars$mpg, 20, "less")

## [1] 18

count_matching(mtcars$mpg, 20, "equal")

## [1] 0

count_matching(mtcars$mpg, 20, "at_least")

## [1] 14

count_matching(mtcars$mpg, 20, "at_most")

## [1] 18

```

Challenge 6: Two-column relationship

Write `column_relationship(df, col_a, col_b)` that takes a data frame and two column names, and returns a named list summarizing the relationship between them:

- `correlation` — the correlation between the two columns (use `cor()` with `use = "complete.obs"` to handle NAs)
- `mean_a` — the mean of `col_a` (NAs removed)
- `mean_b` — the mean of `col_b` (NAs removed)
- `mean_difference` — `mean_a - mean_b`
- `n_complete` — the number of rows where both columns are non-missing

Test it on `mtcars` with "mpg" and "hp". You should see a strong negative correlation (heavier-engined cars get worse mpg).

Original

```
# column_relationship <- function(df, col_a, col_b) {
#   list(
#     correlation = cor(df[[col_a]], df[[col_b]], use = "complete.obs"),
#     mean_a = mean(df[[col_a]], na.rm = T),
#     mean_b = mean(df[[col_b]], na.rm = T),
#     mean_difference = mean_a - mean_b,
#     n_complete = sum(!(is.na(df[[col_a]] & df[[col_b]])))
#   )
# }
```

CORRECTED

```
# takes a data frame and two column names, returns a summary of their relationship
column_relationship <- function(df, col_a, col_b) {
  list(
    # cor() calculates the correlation between two numeric vectors
    # use = "complete.obs" tells cor() to only use rows where BOTH columns have values
    # correlation ranges from -1 (strong negative) to 1 (strong positive)
    correlation = cor(df[[col_a]], df[[col_b]], use = "complete.obs"),
    # mean of each column separately, ignoring NAs
    mean_a = mean(df[[col_a]], na.rm = T),
    mean_b = mean(df[[col_b]], na.rm = T),
    # can't use mean_a - mean_b here because those are just labels in the list, not variables
    # so we have to repeat the mean() calls to do the subtraction
    mean_difference = mean(df[[col_a]], na.rm = T) - mean(df[[col_b]], na.rm = T),
    # complete.cases() returns a logical vector - TRUE for each row where BOTH columns are non-NA
    # sum() counts the TRUEs to give us the number of complete rows
    n_complete = sum(complete.cases(df[[col_a]], df[[col_b]]))
  )
}
```

```
column_relationship(mtcars, "mpg", "hp")
```

```
## $correlation
## [1] -0.7761684
##
## $mean_a
## [1] 20.09062
##
## $mean_b
## [1] 146.6875
##
## $mean_difference
## [1] -126.5969
##
## $n_complete
```


Challenge 7: Multi-column analysis report

Recall your `analyze_column(df, col, threshold = NULL)` function from Challenge 5 of the original practice. Now write `analyze_columns(df, cols, threshold = NULL)` that:

1. Takes a data frame and a **vector** of column names (`cols`).
2. Calls `analyze_column` on each column.
3. Returns a named list where each element is the analysis result for one column.

Test it on `airquality` with `cols = c("Ozone", "Solar.R", "Wind", "Temp")`. Then extract just the `pct_above_threshold` for "Ozone" to confirm the structure works.

Tip: This is a wrapper that applies an existing function repeatedly. You'll need a `for` loop and a results list that grows by name (like in Challenge 2 of the original practice).

```
# analyze_column <- function(df, col, threshold = NULL) {
#   total <- df[[col]]
#   missing <- sum(is.na(df[[col]]))
#   if (is.null(threshold)){
#     threshold <- mean(df[[col]], na.rm = TRUE)
#   }
#   above <- !is.na(total) & total > threshold
#   count <- sum(above)
#   list(
#     column_name = col,
#     n_missing = missing,
#     threshold_used = threshold,
#     n_above_threshold = count,
#     pct_above_threshold = count / (length(total) - missing)
#   )
# }
# analyze_columns <- function(df, cols, threshold = NULL){
#   list()
#   for (cols in colnames(df)) {
#     paste0(df[[col]])
#     analyze_column(df[[col]])
#   }
# }
```

CORRECTED

```
analyze_column <- function(df, col, threshold = NULL) {
  total <- df[[col]]
  missing <- sum(is.na(df[[col]]))
  if (is.null(threshold)){
    threshold <- mean(df[[col]], na.rm = TRUE)
  }
  above <- !is.na(total) & total > threshold
  count <- sum(above)
  list(
    column_name = col,
    n_missing = missing,
    threshold_used = threshold,
```

```

    n_above_threshold = count,
    pct_above_threshold = count / (length(total) - missing)
  )
}

analyze_columns <- function(df, cols, threshold = NULL){
  # start with an empty list to store results - one entry per column
  results <- list()
  # loop over cols (the specific columns passed in, NOT all columns in df)
  # col takes on each column name in turn: "Ozone", "Solar.R", etc.
  for (col in cols) {
    # call analyze_column on each column and store the result
    # results[[col]] creates a named entry - e.g. results[["Ozone"]]
    # pass threshold through so analyze_column can use it (or default to NULL)
    results[[col]] <- analyze_column(df, col, threshold)
  }
  # return the full named list - each element is itself a list from analyze_column
  # access nested results with $ chaining: results$Ozone$pct_above_threshold
  results
}

analyze_columns(airquality, c("Ozone", "Solar.R", "Wind", "Temp"))

```

```

## $Ozone
## $Ozone$column_name
## [1] "Ozone"
##
## $Ozone$n_missing
## [1] 37
##
## $Ozone$threshold_used
## [1] 42.12931
##
## $Ozone$n_above_threshold
## [1] 44
##
## $Ozone$pct_above_threshold
## [1] 0.3793103
##
##
## $Solar.R
## $Solar.R$column_name
## [1] "Solar.R"
##
## $Solar.R$n_missing
## [1] 7
##
## $Solar.R$threshold_used
## [1] 185.9315
##
## $Solar.R$n_above_threshold
## [1] 87
##
## $Solar.R$pct_above_threshold

```

```
## [1] 0.5958904
##
##
## $Wind
## $Wind$column_name
## [1] "Wind"
##
## $Wind$n_missing
## [1] 0
##
## $Wind$threshold_used
## [1] 9.957516
##
## $Wind$n_above_threshold
## [1] 72
##
## $Wind$pct_above_threshold
## [1] 0.4705882
##
##
## $Temp
## $Temp$column_name
## [1] "Temp"
##
## $Temp$n_missing
## [1] 0
##
## $Temp$threshold_used
## [1] 77.88235
##
## $Temp$n_above_threshold
## [1] 85
##
## $Temp$pct_above_threshold
## [1] 0.5555556
analyze_columns(airquality, c("Ozone", "Solar.R", "Wind", "Temp"))$Ozone$pct_above_threshold
## [1] 0.3793103
```

Challenge 8: Formatted summary printer

Write `print_column_report(df, col)` that prints a formatted summary of a column to the console using `cat()`. The output should look something like:

```
Column: Ozone
Mean:    42.13
Median:  31.50
Min:     1.00
Max:    168.00
Missing values: 37
```

Use your `vec_summary()` function internally to compute the stats, then use `cat()` with `"\n"` for line breaks to format the output. `round()` or `sprintf()` can help control the number of decimal places.

Test it on `airquality` with "Ozone" and on `mtcars` with "mpg".

Tip: This function is for side effects (printing), not for returning data. You don't need to return anything meaningful — the printed output *is* the point. But you could also have it `invisible()` return the summary list, so the caller can capture it if they want.

```
#### ORIGINAL
# vec_summary <- function(x) {
#   list(
#     mean = mean(x, na.rm = TRUE),
#     median = median(x, na.rm = TRUE),
#     min = min(x, na.rm = TRUE),
#     max = max(x, na.rm = TRUE),
#     n_missing = sum(is.na(x))
#   )
# }
# print_column_report <- function(df, col) {
#   cat(col, "\n",
#     "Mean" = mean, "\n",
#     "Median" = median, "\n",
#     "Min" = min, "\n",
#     "Max" = max, "\n",
#     "Missing values" = n_missing)
#   vec_summary(df[[col]])
# }
```

```
#### CORRECTED
vec_summary <- function(x) {
  list(
    mean = mean(x, na.rm = TRUE),
    median = median(x, na.rm = TRUE),
    min = min(x, na.rm = TRUE),
    max = max(x, na.rm = TRUE),
    n_missing = sum(is.na(x))
  )
}

print_column_report <- function(df, col) {
  # call vec_summary first and store the result so we can pull stats from it
  stats <- vec_summary(df[[col]])
  # cat() prints to the console - unlike print(), it doesn't add quotes or [1]
  # "\n" adds a line break after each line
  cat("Column", col, "\n")
  # stats$mean accesses the "mean" element from the list that vec_summary returned
  # round(x, 2) limits to 2 decimal places for clean display
  cat(" Mean", round(stats$mean, 2), "\n")
  cat(" Median", round(stats$median, 2), "\n")
  cat(" Min", round(stats$min, 2), "\n")
  cat(" Max", round(stats$max, 2), "\n")
  # n_missing is already a whole number so no need to round
  cat(" Missing values", stats$n_missing, "\n")
  # invisible() returns stats silently - the caller won't see it printed
  # but if they do result <- print_column_report(...), they can access it
  # result$mean will give the full unrounded value (42.12931 not 42.13)
  invisible(stats)
}
```

```

}

print_column_report(airquality, "Ozone")           # just prints

## Column Ozone
## Mean 42.13
## Median 31.5
## Min 1
## Max 168
## Missing values 37

result <- print_column_report(airquality, "Ozone") # prints AND saves

## Column Ozone
## Mean 42.13
## Median 31.5
## Min 1
## Max 168
## Missing values 37

result$mean                                         # access saved stats

## [1] 42.12931

```

Challenge 9: A miniature data cleaning pipeline

Put everything together. Write `clean_and_summarize(df, col, threshold = NULL, drop_outliers = FALSE)` that:

1. Pulls the column from the data frame.
2. Counts the original number of values and the number of NAs.
3. If `drop_outliers` is `TRUE`, removes values that are more than 2 standard deviations from the mean (NAs already removed). Otherwise, keeps all non-NA values.
4. If `threshold` is `NULL`, sets it to the mean of the (cleaned) values.
5. Returns a named list with:
 - `column_name`
 - `n_original` (length of the original column)
 - `n_after_cleaning` (length after removing NAs and possibly outliers)
 - `n_dropped` (the difference)
 - `threshold_used`
 - `n_above_threshold`
 - `summary_stats` — a nested list with `mean`, `median`, `min`, `max` of the cleaned values

Test it on `airquality$Ozone` with default settings, then again with `drop_outliers = TRUE`. The mean should change when outliers are dropped.

Tip: To identify outliers more than 2 SDs from the mean, compute `m <- mean(x, na.rm = TRUE)` and `s <- sd(x, na.rm = TRUE)`, then keep values where `abs(x - m) <= 2 * s`. Remember the `is.na()` check comes first.

```

##### ORIGINAL
# clean_and_summarize <- function(df, col, threshold = NULL, drop_outliers = F) {
#   df[[col]] # pulls the column from df
#   original <- sum(df[[col]]) # counts original number of values
#   missing <- sum(is.na(df[[col]])) # counts the number of NA values
#   m <- mean(df[[col]], na.rm = T) # to identify outliers more than 2 sd from mean

```

```

# s <- sd(df[[col]], na.rm = T)
# if (drop_outliers == T) {
#   keep <- (!(is.na(abs(df[[col]]), na.rm = T))) # remove outliers and save
# } else { # if drop_outliers = F
#   keep <- (!(is.na(df[[col]]))) # save cleaned values
# }
# if (is.null(threshold)) { # if threshold = NULL then set threshold to the
#   threshold <- mean(keep, na.rm = T) # mean of the cleaned values
# }
# sum_keep = sum(keep) # number of cleaned values
# above <- keep > threshold # for use to sum number of values above threshold
# list(
#   column_name = col, # column name
#   n_original = original, # original number of values
#   n_after_cleaning = sum_keep, # number of cleaned values
#   n_dropped = original - sum_keep, # number of values dropped (outliers and NAs)
#   threshold_used = threshold,
#   n_above_threshold = sum(above),
#   summary_stats = list(
#     mean = mean(keep),
#     median = median(keep),
#     min = min(keep),
#     max = max(keep)
#   )
# )
# }

```

CORRECTED

```

# takes a data frame, column name, optional threshold, and outlier toggle
# cleans the column and returns a detailed summary
clean_and_summarize <- function(df, col, threshold = NULL, drop_outliers = FALSE) {
  # WRONG: df[[col]] on its own line does nothing - must store it in a variable
  # RIGHT: save the column as x so we can reference it throughout the function
  x <- df[[col]]
  # WRONG: sum(df[[col]]) adds up all the values (e.g. 5+8+12...)
  # RIGHT: length() counts how many elements are in the vector
  original <- length(x)
  # this was correct - sum(is.na()) counts the NAs
  missing <- sum(is.na(x))
  # these were correct - compute mean and sd for outlier detection
  # calculated on the full column (ignoring NAs) before any cleaning
  m <- mean(x, na.rm = TRUE)
  s <- sd(x, na.rm = TRUE)
  # WRONG: if (drop_outliers = T) - single = is assignment, not comparison
  # RIGHT: if (drop_outliers) works because it's already TRUE or FALSE
  if (drop_outliers) {
    # WRONG: is.na(abs(df[[col]], na.rm = T)) - na.rm doesn't go in abs()
    # and is.na() gives TRUE/FALSE, not the actual values
    # RIGHT: two conditions combined with &
    # !is.na(x) - keep non-missing values
    # abs(x - m) <= 2 * s - keep values within 2 standard deviations of the mean
    # abs() gives the distance from the mean regardless of direction
    # this returns the actual VALUES that pass both tests, not TRUE/FALSE
    clean <- x[!is.na(x) & abs(x - m) <= 2 * s]
  }
}

```

```

} else {
  # WRONG: !is.na(df[[col]]) gives a logical vector (TRUE/FALSE)
  # RIGHT: x[!is.na(x)] gives the actual non-NA values
  clean <- x[!is.na(x)]
}
# this logic was correct - if no threshold given, default to mean of cleaned values
if (is.null(threshold)) {
  # WRONG: mean(keep, na.rm = T) - keep was a logical vector, not values
  # RIGHT: mean(clean) - clean holds the actual values, no na.rm needed
  #         because NAs were already removed above
  threshold <- mean(clean)
}
# WRONG: sum(keep) counts TRUEs in a logical vector, not the number of values
# RIGHT: length(clean) counts how many values survived cleaning
n_clean <- length(clean)
# WRONG: keep > threshold compared TRUE/FALSE against a number
# RIGHT: clean > threshold compares actual values against threshold
#         sum() counts how many are above
above <- sum(clean > threshold)
list(
  column_name = col,
  # WRONG: original was sum() of values, not a count
  # RIGHT: now it's length(), so this works correctly
  n_original = original,
  n_after_cleaning = n_clean,
  # difference tells us how many were removed (NAs + any outliers)
  n_dropped = original - n_clean,
  threshold_used = threshold,
  n_above_threshold = above,
  # nested list for summary statistics
  summary_stats = list(
    # WRONG: mean(keep) was averaging TRUE/FALSE values
    # RIGHT: mean(clean) averages the actual cleaned numbers
    # no na.rm needed - NAs were already stripped out
    mean = mean(clean),
    median = median(clean),
    min = min(clean),
    max = max(clean)
  )
)
}

clean_and_summarize(airquality, "Ozone")

```

```

## $column_name
## [1] "Ozone"
##
## $n_original
## [1] 153
##
## $n_after_cleaning
## [1] 116
##
## $n_dropped

```

```

## [1] 37
##
## $threshold_used
## [1] 42.12931
##
## $n_above_threshold
## [1] 44
##
## $summary_stats
## $summary_stats$mean
## [1] 42.12931
##
## $summary_stats$median
## [1] 31.5
##
## $summary_stats$min
## [1] 1
##
## $summary_stats$max
## [1] 168

clean_and_summarize(airquality, "Ozone", drop_outliers = TRUE)

## $column_name
## [1] "Ozone"
##
## $n_original
## [1] 153
##
## $n_after_cleaning
## [1] 110
##
## $n_dropped
## [1] 43
##
## $threshold_used
## [1] 37.44545
##
## $n_above_threshold
## [1] 42
##
## $summary_stats
## $summary_stats$mean
## [1] 37.44545
##
## $summary_stats$median
## [1] 29.5
##
## $summary_stats$min
## [1] 1
##
## $summary_stats$max
## [1] 108

```


Challenge 10: Bonus — functions that take functions

This is a step toward how `lapply()` and friends work (which is the next chapter). Write `summarize_with(x, func, ...)` that takes a vector `x`, a function `func`, and any extra arguments via `...`. It should call `func(x, ...)` and return the result.

So `summarize_with(airquality$Ozone, mean, na.rm = TRUE)` would call `mean(airquality$Ozone, na.rm = TRUE)`. And `summarize_with(mtcars$mpg, median)` would call `median(mtcars$mpg)`.

This might feel pointless — why wrap a function call inside another function call? The power is that *the function itself becomes an argument*, so the same wrapper can compute means, medians, sums, or anything else without you rewriting it.

Test it with at least three different functions: `mean`, `median`, and `sd`.

```
#### got it right
summarize_with <- function(x, func, ...) {
  result <- func(x, ...)
  result
}

summarize_with(airquality$Ozone, mean, na.rm = TRUE)

## [1] 42.12931

summarize_with(mtcars$mpg, median)

## [1] 19.2

summarize_with(mtcars$mpg, sd)

## [1] 6.026948
```